Lista dwukierunkowa

Generated by Doxygen 1.18.0

1 Directory Hierarchy 1

1.1 Directories . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Directory Documentation 7

4.1 kod Directory Reference . . . 7

5 Class Documentation 9

5.1 DoublyLinkedList Class Reference . . . 9

5.1.1 Detailed Description . . . 9

5.1.2 Constructor & Destructor Documentation . . . 9

5.1.2.1 DoublyLinkedList() . . . 9

5.1.2.2 ~DoublyLinkedList() . . . 10

5.1.3 Member Function Documentation . . . 10

5.1.3.1 clear() . . . 10

5.1.3.2 get_size() . . . 10

5.1.3.3 getIterator() . . . 10

5.1.3.4 insert_at() . . . 10

5.1.3.5 pop_back() . . . 10

5.1.3.6 pop_front() . . . 10

5.1.3.7 print() . . . 10

5.1.3.8 print_reverse() . . . 11

5.1.3.9 push_back() . . . 11

5.1.3.10 push_front() . . . 11

5.1.3.11 remove_at() . . . 11

5.2 ListIterator Class Reference . . . 11

5.2.1 Detailed Description . . . 12

5.2.2 Constructor & Destructor Documentation . . . 12

5.2.2.1 ListIterator() . . . 12

5.2.3 Member Function Documentation . . . 12

5.2.3.1 isValid() . . . 12

5.2.3.2 next() . . . 12

5.2.3.3 prev() . . . 12

5.2.3.4 printCurrent() . . . 13

5.3 Node Class Reference . . . 13

5.3.1 Detailed Description . . . 13

5.3.2 Constructor & Destructor Documentation . . . 14

5.3.2.1 Node() . . . 14

5.3.3 Member Data Documentation . . . 14

5.3.3.1 data . . . 14
5.3.3.2 next . . . 14
5.3.3.3 prev . . . 14
5.4 NodeFactory Class Reference . . . 14
5.4.1 Detailed Description . . . 15
5.4.2 Member Function Documentation . . . 15
5.4.2.1 createNode() . . . 15

6 File Documentation **17**
6.1 kod/DoublyLinkedList.cpp File Reference . . . 17
6.2 kod/DoublyLinkedList.h File Reference . . . 17
6.3 DoublyLinkedList.h . . . 17
6.4 kod/ListIterator.cpp File Reference . . . 18
6.5 kod/Listiterator.h File Reference . . . 18
6.6 Listiterator.h . . . 18
6.7 kod/main.cpp File Reference . . . 19
6.7.1 Function Documentation . . . 19
6.7.1.1 main() . . . 19
6.8 kod/Node.cpp File Reference . . . 19
6.9 kod/Node.h File Reference . . . 19
6.10 Node.h . . . 19
6.11 kod/NodeFactory.cpp File Reference . . . 20
6.12 kod/NodeFactory.h File Reference . . . 20
6.13 NodeFactory.h . . . 20

Index **21**

Chapter 1

Directory Hierarchy

1.1 Directories

kod 7
DoublyLinkedList.cpp 17
DoublyLinkedList.h 17
ListIterator.cpp 18
Listiterator.h 18
main.cpp 19
Node.cpp 19
Node.h 19
NodeFactory.cpp 20
NodeFactory.h 20

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DoublyLinkedList
: Główna klasa zarządzająca listą dwukierunkową 9

ListIterator
: Klasa realizująca wzorzec projektowy Iterator. Pozwala na poruszanie się po elementach listy dwukierunkowej bez odsłaniania jej wewnętrznej struktury 11

Node
: Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej 13

NodeFactory
: Klasa realizująca wzorzec projektowy Factory (Fabryka). Odpowiada za bezpieczne tworzenie nowych obiektów typu Node 14

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

kod/DoublyLinkedList.cpp 17
kod/DoublyLinkedList.h 17
kod/ListIterator.cpp 18
kod/Listiterator.h 18
kod/main.cpp 19
kod/Node.cpp 19
kod/Node.h 19
kod/NodeFactory.cpp 20
kod/NodeFactory.h 20

Chapter 4

Directory Documentation

4.1 kod Directory Reference

Files

- file DoublyLinkedList.cpp
- file DoublyLinkedList.h
- file ListIterator.cpp
- file Listiterator.h
- file main.cpp
- file Node.cpp
- file Node.h
- file NodeFactory.cpp
- file NodeFactory.h

Chapter 5

Class Documentation

5.1 DoublyLinkedList Class Reference

Główna klasa zarządzająca listą dwukierunkową.

`#include <DoublyLinkedList.h>`

Public Member Functions

- DoublyLinkedList ()
- ~DoublyLinkedList ()
- void push_front (int data)
- void push_back (int data)
- void insert_at (int index, int data)
- void pop_front ()
- void pop_back ()
- void remove_at (int index)
- void print ()
- void print_reverse ()
- void clear ()
- int get_size ()
- ListIterator getIterator ()

  Zwraca iterator ustawiony na początek listy.

5.1.1 Detailed Description

Główna klasa zarządzająca listą dwukierunkową.

5.1.2 Constructor & Destructor Documentation

5.1.2.1 DoublyLinkedList()

```
DoublyLinkedList::DoublyLinkedList ()
```

5.1.2.2 ~DoublyLinkedList()

```
DoublyLinkedList::~DoublyLinkedList ()
```

5.1.3 Member Function Documentation

5.1.3.1 clear()

```
void DoublyLinkedList::clear ()
```

5.1.3.2 get_size()

```
int DoublyLinkedList::get_size ()
```

5.1.3.3 getIterator()

```
ListIterator DoublyLinkedList::getIterator ()
```

Zwraca iterator ustawiony na początek listy.

Returns

Obiekt ListIterator

5.1.3.4 insert_at()

```
void DoublyLinkedList::insert_at (
            int index,
            int data)
```

5.1.3.5 pop_back()

```
void DoublyLinkedList::pop_back ()
```

5.1.3.6 pop_front()

```
void DoublyLinkedList::pop_front ()
```

5.1.3.7 print()

```
void DoublyLinkedList::print ()
```

5.1.3.8 print_reverse()

```
void DoublyLinkedList::print_reverse ()
```

5.1.3.9 push_back()

```
void DoublyLinkedList::push_back (
            int data)
```

5.1.3.10 push_front()

```
void DoublyLinkedList::push_front (
            int data)
```

5.1.3.11 remove_at()

```
void DoublyLinkedList::remove_at (
            int index)
```

The documentation for this class was generated from the following files:

- kod/DoublyLinkedList.h
- kod/DoublyLinkedList.cpp

5.2 ListIterator Class Reference

Klasa realizująca wzorzec projektowy Iterator. Pozwala na poruszanie się po elementach listy dwukierunkowej bez odsłaniania jej wewnętrznej struktury.

```
#include <Listiterator.h>
```

Public Member Functions

- ListIterator (Node *startNode)

  Konstruktor iteratora.
- void next ()

  Przesuwa iterator na następny element.
- void prev ()

  Przesuwa iterator na poprzedni element.
- void printCurrent ()

  Zwraca wartość aktualnego węzła i wypisuje ją na ekran.
- bool isValid ()

  Sprawdza, czy iterator doszedł do końca/początku (czy wskazuje na nullptr).

5.2.1 Detailed Description

Klasa realizująca wzorzec projektowy Iterator. Pozwala na poruszanie się po elementach listy dwukierunkowej bez odsłaniania jej wewnętrznej struktury.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 ListIterator()

```
ListIterator::ListIterator (
            Node * startNode)
```

Konstruktor iteratora.

Parameters

startNode	Węzeł, od którego rozpoczynamy iterację

5.2.3 Member Function Documentation

5.2.3.1 isValid()

```
bool ListIterator::isValid ()
```

Sprawdza, czy iterator doszedł do końca/początku (czy wskazuje na nullptr).

Returns

true jeśli wskazuje na poprawny obiekt, false jeśli na nullptr

5.2.3.2 next()

```
void ListIterator::next ()
```

Przesuwa iterator na następny element.

5.2.3.3 prev()

```
void ListIterator::prev ()
```

Przesuwa iterator na poprzedni element.

5.2.3.4 printCurrent()

```
void ListIterator::printCurrent ()
```

Zwraca wartość aktualnego węzła i wypisuje ją na ekran.

The documentation for this class was generated from the following files:

- kod/Listiterator.h
- kod/ListIterator.cpp

5.3 Node Class Reference

Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej.

```
#include <Node.h>
```

Public Member Functions

- Node (int val)

  Konstruktor węzła.

Public Attributes

- int data

  Przechowywana wartość całkowita.
- Node * prev

  Wskaźnik na poprzedni element.
- Node * next

  Wskaźnik na następny element.

5.3.1 Detailed Description

Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 Node()

```
Node::Node (
            int val)
```

Konstruktor węzła.

Parameters

val	Wartość do przechowania w węźle

5.3.3 Member Data Documentation

5.3.3.1 data

```
int Node::data
```

Przechowywana wartość całkowita.

5.3.3.2 next

```
Node* Node::next
```

Wskaźnik na następny element.

5.3.3.3 prev

```
Node* Node::prev
```

Wskaźnik na poprzedni element.

The documentation for this class was generated from the following files:

- kod/Node.h
- kod/Node.cpp

5.4 NodeFactory Class Reference

Klasa realizująca wzorzec projektowy Factory (Fabryka). Odpowiada za bezpieczne tworzenie nowych obiektów typu Node.

```
#include <NodeFactory.h>
```

Static Public Member Functions

- static Node * createNode (int data)

  Metoda fabrykująca nowy węzeł.

5.4.1 Detailed Description

Klasa realizująca wzorzec projektowy Factory (Fabryka). Odpowiada za bezpieczne tworzenie nowych obiektów typu Node.

5.4.2 Member Function Documentation

5.4.2.1 createNode()

```
Node * NodeFactory::createNode (
            int data)  [static]
```

Metoda fabrykująca nowy węzeł.

Parameters

data	Wartość, która ma zostać przypisana do węzła.

Returns

Wskaźnik na nowo utworzony obiekt Node na stercie.

The documentation for this class was generated from the following files:

- kod/NodeFactory.h
- kod/NodeFactory.cpp

Chapter 6

File Documentation

6.1 kod/DoublyLinkedList.cpp File Reference

```
#include "DoublyLinkedList.h"
#include "NodeFactory.h"
#include <iostream>
```

6.2 kod/DoublyLinkedList.h File Reference

```
#include "Node.h"
#include "ListIterator.h"
```

Classes

- class DoublyLinkedList

  Główna klasa zarządzająca listą dwukierunkową.

6.3 DoublyLinkedList.h

Go to the documentation of this file.

```
00001 #ifndef DOUBLYLINKEDLIST_H
00002 #define DOUBLYLINKEDLIST_H
00003 #include "Node.h"
00004 #include "ListIterator.h"
00005
00010 class DoublyLinkedList {
00011 private:
00012     Node* head;
00013     Node* tail;
00014     int size;
00015
00016 public:
00017     DoublyLinkedList();
00018     ~DoublyLinkedList();
00019
00020     void push_front(int data);
00021     void push_back(int data);
```

```
00022     void insert_at(int index, int data);
00023
00024     void pop_front();
00025     void pop_back();
00026     void remove_at(int index);
00027
00028     void print();
00029     void print_reverse();
00030     void clear();
00031     int get_size();
00032
00037     ListIterator getIterator();
00038 };
00039
00040 #endif
```

6.4 kod/ListIterator.cpp File Reference

```
#include "ListIterator.h"
```

6.5 kod/Listiterator.h File Reference

```
#include "Node.h"
#include <iostream>
```

Classes

- class ListIterator

  Klasa realizująca wzorzec projektowy Iterator. Pozwala na poruszanie się po elementach listy dwukierunkowej bez odsłaniania jej wewnętrznej struktury.

6.6 Listiterator.h

Go to the documentation of this file.

```
00001 #ifndef LISTITERATOR_H
00002 #define LISTITERATOR_H
00003 #include "Node.h"
00004 #include <iostream>
00005
00011 class ListIterator {
00012 private:
00013     Node* current;
00014
00015 public:
00020     ListIterator(Node* startNode);
00021
00025     void next();
00026
00030     void prev();
00031
00035     void printCurrent();
00036
00041     bool isValid();
00042 };
00043
00044 #endif
```

6.7 kod/main.cpp File Reference

```
#include <iostream>
#include "DoublyLinkedList.h"
```

Functions

- int main ()

6.7.1 Function Documentation

6.7.1.1 main()

```
int main ()
```

6.8 kod/Node.cpp File Reference

```
#include "Node.h"
```

6.9 kod/Node.h File Reference

Classes

- class Node

  Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej.

6.10 Node.h

Go to the documentation of this file.

```
00001 #ifndef NODE_H
00002 #define NODE_H
00003
00008 class Node {
00009 public:
00010     int data;
00011     Node* prev;
00012     Node* next;
00013
00018     Node(int val);
00019 };
00020
00021 #endif
```

6.11 kod/NodeFactory.cpp File Reference

```
#include "NodeFactory.h"
```

6.12 kod/NodeFactory.h File Reference

```
#include "Node.h"
```

Classes

- class NodeFactory

  Klasa realizująca wzorzec projektowy Factory (Fabryka). Odpowiada za bezpieczne tworzenie nowych obiektów typu Node.

6.13 NodeFactory.h

Go to the documentation of this file.

```
00001 #ifndef NODEFACTORY_H
00002 #define NODEFACTORY_H
00003 #include "Node.h"
00004
00010 class NodeFactory {
00011 public:
00017     static Node* createNode(int data);
00018 };
00019
00020 #endif
```

Index

~DoublyLinkedList
    DoublyLinkedList, 9

clear
    DoublyLinkedList, 10
createNode
    NodeFactory, 15

data
    Node, 14
DoublyLinkedList, 9
    ~DoublyLinkedList, 9
    clear, 10
    DoublyLinkedList, 9
    get_size, 10
    getIterator, 10
    insert_at, 10
    pop_back, 10
    pop_front, 10
    print, 10
    print_reverse, 10
    push_back, 11
    push_front, 11
    remove_at, 11

get_size
    DoublyLinkedList, 10
getIterator
    DoublyLinkedList, 10

insert_at
    DoublyLinkedList, 10
isValid
    ListIterator, 12

kod Directory Reference, 7
kod/DoublyLinkedList.cpp, 17
kod/DoublyLinkedList.h, 17
kod/ListIterator.cpp, 18
kod/Listiterator.h, 18
kod/main.cpp, 19
kod/Node.cpp, 19
kod/Node.h, 19
kod/NodeFactory.cpp, 20
kod/NodeFactory.h, 20

ListIterator, 11
    isValid, 12
    ListIterator, 12
    next, 12
    prev, 12
    printCurrent, 12

main
    main.cpp, 19
main.cpp
    main, 19

next
    ListIterator, 12
    Node, 14
Node, 13
    data, 14
    next, 14
    Node, 14
    prev, 14
NodeFactory, 14
    createNode, 15

pop_back
    DoublyLinkedList, 10
pop_front
    DoublyLinkedList, 10
prev
    ListIterator, 12
    Node, 14
print
    DoublyLinkedList, 10
print_reverse
    DoublyLinkedList, 10
printCurrent
    ListIterator, 12
push_back
    DoublyLinkedList, 11
push_front
    DoublyLinkedList, 11

remove_at
    DoublyLinkedList, 11